AVISO: Diretrizes do Projeto PhotoCensus (Módulos 1 e 2)

Prezados alunos,

Este documento reúne todas as orientações do projeto **PhotoCensus**, que será desenvolvido em **grupos de até 3 pessoas** e avaliado em duas apresentações: o **Módulo 1 (17/11)**, mais curto e com orientações, vale **40% da nota**, e o **Módulo 2 (24/11)**, mais longo e completo, vale **60% da nota**. Fiquem atentos aos requisitos, às regras de entrega e à dinâmica das apresentações em laboratório:

🔎 Sobre o Projeto PhotoCensus

O **PhotoCensus** é um sistema em **console/prompt** que faz o "censo" dos objetos que aparecem em fotos: quantas pessoas, carros, cadeiras, cachorros etc. existem em cada imagem, em cada pasta e no acervo inteiro. A detecção dos objetos é feita por um modelo de Inteligência Artificial (YOLO) usado como **caixa-preta**: vocês não precisam (e não vão) estudar como a IA funciona. O trabalho do grupo é construir toda a lógica do sistema em Python em cima dos resultados que a caixa-preta devolve: no Módulo 1 com **if, for e while** e no Módulo 2 também com **estruturas de dados**.

👥 Formação dos Grupos e Monitores

- Os grupos devem ter **no máximo 3 integrantes**.
- Até o dia **08/11**, cada grupo deve preencher a planilha online com o nome dos integrantes, o **monitor** do grupo e o link do fork no GitHub: https://docs.google.com/spreadsheets/d/138nGkNHfpD6wukGm2NN8GTbmOv1dvU3zvQ13B8fQ3tQ/edit.
- O número do grupo usado em todas as entregas (slides, pasta no GitHub e Pull Request) será o da linha preenchida na planilha (ex.: Grupo_03).

📦 Kit do Projeto (Fotos e Caixa-Preta)

- **Onde fica:** o kit está na pasta /kit do repositório central (git@github.com:MarcondesRicarte/PhotoCensus-UPE.git) e já vem junto com o fork. **Não copiem o kit** para a pasta do grupo.
- **Fotos:** 48 fotos organizadas em 6 pastas com 8 fotos cada: animals, kitchen, living_room, office, sports e street.
- **Caixa-preta:** o arquivo kit/black_box.py é importado pelas linhas iniciais do main.py do grupo (ver a seção seguinte). Ele **não pode ser alterado**.
- **Instalação:** para as fotos do kit **não é preciso instalar nada** (basta Python 3.9 ou mais recente). Na raiz do repositório, rodem python kit/test_kit.py para conferir. O instalador automático (kit/install_windows.bat ou kit/install_linux_mac.sh) só é necessário para analisar fotos novas, que é opcional.
- **Documentação:** o arquivo kit/README.md traz todas as funções, exemplos de uso e a lista de classes (objetos) que aparecem nas fotos.

📁 Estrutura do Repositório

O repositório central é organizado assim. Cada grupo trabalha **somente na própria pasta** dentro de /groups/:

```
PhotoCensus-UPE/
├── README.md           <- instruções gerais
├── docs/               <- este documento
├── kit/                <- kit do aluno (NÃO ALTERAR)
│   ├── black_box.py
│   ├── test_kit.py, example_module1.py, example_module2.py
│   ├── install.py, install_windows.bat, install_linux_mac.sh
│   ├── README.md, IMAGE_CREDITS.txt
│   ├── data/
│   └── images/         <- animals, kitchen, living_room, office,
│                          sports, street
└── groups/
    ├── group_00/       <- modelo: copiem para group_XX
    │   ├── main.py
    │   └── README.md
    ├── group_01/       <- código do Grupo 01
    └── ...
```

- Copiem a pasta modelo /groups/group_00/ para /groups/group_XX/ (XX = número do grupo na planilha) e preencham o README.md dela com os integrantes e o monitor.
- O código do grupo fica em /groups/group_XX/main.py (e em outros arquivos .py, se quiserem). O modelo já começa com as linhas que encontram a caixa-preta; **não as removam**:

```
import os
import sys

GROUP_DIR = os.path.dirname(os.path.abspath(__file__))
sys.path.append(os.path.join(GROUP_DIR, "..", "..", "kit"))
from black_box import *
```

- Para rodar, na raiz do repositório: python groups/group_XX/main.py.
- Fotos próprias (extra opcional) ficam em /groups/group_XX/my_photos/, nunca em /kit/images/.

🌐 Regras Gerais (valem para os dois módulos)

- **Sugestão de idioma:** o kit é todo em inglês e **recomendamos** (não é obrigatório) que o código do grupo também seja: nomes de arquivos, variáveis, funções e comentários (ex.: main.py, photo_count, show_menu()). Também é aceito código em português.
- **Interface:** toda a interface deve ser em **console** (nada de interface gráfica).
- **Kit intocado:** nada dentro de /kit pode ser alterado, e cada grupo só altera a própria pasta /groups/group_XX/.
- **Cálculos com laços:** as funções prontas proibidas em cada módulo estão indicadas nos requisitos; contagens, somas, máximos e mínimos devem ser feitos pelo grupo.

📅 Calendário

- **08/11:** prazo para preencher a planilha de grupos e monitores.
- **16/11, 18h:** prazo dos slides e do commit do Módulo 1.
- **17/11:** apresentação do Módulo 1 (40% da nota).
- **23/11, 18h:** prazo dos slides e do commit do Módulo 2.
- **24/11:** apresentação do Módulo 2 (60% da nota).
- Em cada apresentação, a ordem de defesa será definida por **sorteio 1 hora antes** do início da aula, no **laboratório**. Todos os grupos devem estar prontos para apresentar.

Módulo 1 – Apresentação de 17/11 (40% da nota)

O Módulo 1 é uma apresentação **curta e de acompanhamento**. Além de avaliar o que foi feito, ela serve para eu dar orientações a cada grupo para o Módulo 2.

🧰 Funções da Caixa-Preta Liberadas no Módulo 1

No Módulo 1 vocês usarão somente funções que devolvem **valores simples** (int, float, str ou bool). Por isso, **não é exigido o uso de estruturas de dados** (listas, tuplas, dicionários ou conjuntos): todos os requisitos podem ser resolvidos com variáveis, condicionais e laços.

- folder_exists(folder) → True/False, ótima para validar entradas com while.
- total_photos(folder) → quantidade de fotos da pasta.
- photo_name(folder, number) → caminho da foto (os números começam em 1).
- count_objects(photo, min_confidence) → total de objetos detectados na foto.
- count_class(photo, class_name, min_confidence) → quantos objetos de uma classe (ex.: "person").
- main_object(photo) e max_confidence(photo) → objeto com maior confiança e o valor dessa confiança (0.0 a 1.0).
- show_folders() e show_classes() → apenas imprimem as opções disponíveis.

✅ Requisitos do Módulo 1

1. **Menu principal em laço:** o programa deve exibir um menu numerado dentro de um **while**, repetir até o usuário escolher a opção **0 - Exit** e tratar opções inválidas com mensagem de erro.
2. **Validação de entradas com while:** toda entrada deve ser pedida novamente até ser válida: nome da pasta (usando folder_exists), número da foto (entre 1 e o total da pasta) e confiança mínima (entre 0 e 1).
3. **Configurar a confiança mínima:** opção do menu para alterar a confiança mínima (valor inicial 0.5), que deve ser usada em todas as outras opções.
4. **Analisar uma foto:** o usuário escolhe a pasta e o número da foto; o sistema mostra o caminho, o total de objetos, o objeto principal e a maior confiança em porcentagem (ex.: 89%).
5. **Contar uma classe na foto:** o usuário digita uma classe (ex.: car) e o sistema informa quantos objetos dela existem na foto escolhida.
6. **Relatório da pasta:** com um **for** percorrendo todas as fotos da pasta, mostrar uma linha por foto com a quantidade de objetos e, ao final, o total geral, a média de objetos por foto e quantas fotos ficaram sem nenhum objeto.
7. **Foto mais cheia e mais vazia:** indicar a foto com mais objetos e a foto com menos objetos da pasta, fazendo a busca do maior e do menor manualmente, com variáveis e if dentro do laço.
8. **Procurar uma classe na pasta:** informar em quantas fotos a classe aparece, a quantidade total encontrada e a porcentagem de fotos da pasta em que ela aparece.
9. **Gráfico de barras em texto:** para cada foto da pasta, imprimir uma barra com um # por objeto (ex.: street_04 | ########## (11)), usando **laços aninhados**.

Proibido no Módulo 1: sum(), max() e min(). O código principal deve ficar em groups/group_XX/main.py, com nomes significativos e comentários nos trechos principais.

⏱️ Tempo e Dinâmica da Defesa

- Cada grupo terá o tempo total e improrrogável de **15 minutos**.
- **Divisão do tempo:** 10 minutos para a demonstração do sistema + até 5 minutos para perguntas e **orientações do professor para o Módulo 2**.

- A avaliação e a nota serão **individualizadas** com base no domínio técnico demonstrado por cada componente na explicação do código e resolução de testes rápidos na hora (ex.: “mostre o relatório da pasta animals com confiança 0.7”).

📊 Critérios de Avaliação (40% da nota do projeto)

- **Requisitos funcionando (4,0 pontos):** cada requisito do Módulo 1 rodando corretamente no console.
- **Uso correto de laços e validações (2,0 pontos):** escolha adequada entre for e while, acumuladores, contadores e validação das entradas.
- **Domínio individual (3,0 pontos):** explicação do código e resposta aos testes rápidos.
- **Organização do código (1,0 ponto):** legibilidade, nomes significativos e comentários.

🖥️ O que deve ser apresentado e entregue?

1. **Slides dos Requisitos:** Um dos componentes do grupo deve anexar no Classroom o arquivo de slides até **18h** do dia **16/11**. O nome do arquivo deve ser obrigatoriamente o número do grupo (Ex: Grupo_03.pdf). Usem **no máximo 6 slides**, focando em **quais** requisitos foram implementados e **como** foram implementados.
2. **Código Rodando:** Cada grupo deve apresentar o sistema funcionando na prática (via **console/prompt**).
    - ⚠️ ATENÇÃO: Caso o grupo **não possua notebook** para apresentar, avise-me por mensagem privada imediatamente para que eu possa baixar o código do GitHub com antecedência e deixá-lo pronto na minha máquina.
3. **Prazo Limite do Commit:** Todos os códigos devem estar commitados nos repositórios até as **18h (16/11)**. Commits realizados após esse horário não serão contabilizados para a nota desta etapa.

Módulo 2 – Apresentação de 24/11 (60% da nota)

O Módulo 2 é a apresentação **completa** do projeto. O sistema deixa de analisar uma pasta por vez e passa a trabalhar com o **acervo inteiro**, organizando os resultados da IA em **estruturas de dados**.

🔎 O que muda no Módulo 2

- O kit continua o mesmo (mesmas fotos e mesmo black_box.py). Agora ficam liberadas também as funções que devolvem estruturas de dados:
    - list_folders() → lista com os nomes das 6 pastas.
    - list_photos(folder) → lista com os caminhos das fotos da pasta.
    - detect(photo) → lista de tuplas (classe, confiança), ex.: [("car", 0.70), ("person", 0.41)]. Ela devolve tudo a partir de 0.25 de confiança: filtrar é tarefa do grupo.
- As funcionalidades do Módulo 1 devem continuar funcionando, podendo ser reescritas com as novas estruturas.
- As **orientações recebidas no dia 17/11** devem estar incorporadas ao código e serão verificadas.

🧱 Estruturas de Dados Obrigatórias

- **Dicionário de resultados:** {photo_path: [(class_name, confidence), ...]}, montado uma única vez ao iniciar o programa, com todas as fotos de todas as pastas.
- **Listas e tuplas:** detecções de cada foto, listas filtradas e o ranking de classes [(class_name, amount), ...].

- **Dicionário de contagem:** {class_name: amount} (inventário).
- **Dicionário de índice reverso:** {class_name: [photos where it appears]}.
- **Conjuntos (set):** classes distintas de cada pasta, para comparações.
- **Matriz (lista de listas):** tabela com as pastas nas linhas e as classes nas colunas.

✅ Requisitos do Módulo 2

1. **Carga do acervo:** ao iniciar, percorrer todas as pastas e fotos com laços aninhados, montar o dicionário de resultados e mostrar um resumo (quantidade de pastas, de fotos e de detecções).
2. **Filtro de confiança:** criar uma função que recebe uma lista de detecções e a confiança mínima e devolve uma nova lista filtrada. Ao mudar a confiança no menu (validada com while), todos os resultados devem respeitar o novo valor.
3. **Inventário:** exibir o dicionário de contagem de objetos do acervo inteiro ou de uma pasta escolhida pelo usuário.
4. **Ranking Top N:** converter o inventário em lista de tuplas e ordená-la em ordem decrescente com um algoritmo implementado pelo grupo (bolha, seleção ou inserção); exibir as N classes mais frequentes (N validado).
5. **Busca reversa:** dada uma classe, listar todas as fotos em que ela aparece e a quantidade em cada uma, usando o índice reverso; informar quando a classe não for encontrada.
6. **Comparação de pastas com conjuntos:** dadas duas pastas, mostrar as classes em comum, as exclusivas de cada uma e a união.
7. **Matriz pasta × classe:** imprimir a matriz como tabela alinhada, com o total de cada linha e de cada coluna, e indicar a combinação (pasta, classe) com o maior valor.
8. **Histograma em texto:** imprimir o inventário como gráfico de barras com #, na ordem do ranking.
9. **Relatório em arquivo:** salvar o inventário e o ranking em report.txt usando open(), e ter uma opção para ler e exibir o arquivo salvo.
10. **Modularização:** código organizado em funções com parâmetros e retorno; o main.py deve conter basicamente o menu. É permitido separar as funções em outros arquivos .py.

Proibido no Módulo 2: sorted(), .sort(), max(), min(), sum(), collections.Counter, pandas e numpy (ordenação, contagens, máximos e somas devem ser implementados com laços).

Extras (até +1,0 ponto, limitado à nota 10): analisar fotos tiradas pelo próprio grupo, guardadas em /groups/group_XX/my_photos/ (usando o instalador do kit); exportar o inventário em CSV; ou indicar a foto “mais parecida” com uma foto escolhida, comparando os conjuntos de classes.

⏱️ Tempo e Dinâmica da Defesa

- Cada grupo terá o tempo total e improrrogável de **20 minutos**.
- **Divisão do tempo:** 15 minutos para a apresentação do grupo + até 5 minutos para arguição e perguntas do professor.
- Todos os componentes devem apresentar uma parte do código. O professor poderá pedir pequenas alterações na hora (ex.: “mude o ranking para ordem crescente” ou “mostre a busca reversa de person com confiança 0.8”).
- A avaliação e a nota serão **individualizadas** com base no domínio técnico demonstrado por cada componente na explicação do código e resolução de testes rápidos na hora.

📊 Critérios de Avaliação (60% da nota do projeto)

- **Requisitos funcionando (4,0 pontos):** cada requisito do Módulo 2 rodando corretamente no console.
- **Uso correto das estruturas de dados (2,0 pontos):** escolha adequada de listas, tuplas, dicionários, conjuntos e matriz, e implementação manual da ordenação.
- **Domínio individual (3,0 pontos):** explicação do código e resolução das alterações pedidas na hora.
- **Organização e evolução (1,0 ponto):** modularização em funções, legibilidade e incorporação das orientações do Módulo 1.

🖥️ O que deve ser apresentado e entregue?

1. **Slides dos Requisitos:** Um dos componentes do grupo deve anexar no Classroom o arquivo de slides até **18h** do dia **23/11**. O nome do arquivo deve ser obrigatoriamente o número do grupo (Ex: Grupo_03.pdf). Os slides devem focar em **quais** requisitos foram implementados e **como** foram implementados, incluindo um esquema das estruturas de dados utilizadas.
2. **Código Rodando:** Cada grupo deve apresentar o sistema funcionando na prática (via **console/prompt**).
    - ⚠️ ATENÇÃO: Caso o grupo **não possua notebook** para apresentar, avise-me por mensagem privada imediatamente para que eu possa baixar o código do GitHub com antecedência e deixá-lo pronto na minha máquina.
3. **Prazo Limite do Commit:** Todos os códigos devem estar commitados nos repositórios até as **18h (23/11)**. Commits realizados após esse horário não serão contabilizados para a nota desta etapa.

🐙🛠️ Unificação dos Repositórios no GitHub (Instruções para os Grupos)

Para centralizarmos o ecossistema do projeto **PhotoCensus** e gerarmos uma vitrine pública do desempenho da turma, vamos unificar os projetos no meu GitHub de duas formas combinadas. Sigam os passos abaixo:

Passo 1: O Fork do Repositório Central

- Acessem o repositório central do projeto no meu perfil: git@github.com:MarcondesRicarte/PhotoCensus-UPE.git.
- Um dos integrantes do grupo deve clicar no botão **Fork** (no canto superior direito) para gerar uma cópia do projeto para o perfil do grupo.
- Configurem o repositório como **Público** (essencial para a visibilidade de vocês e do projeto) e informem o link na planilha de grupos.
- Clonem o fork na máquina de vocês e rodem python kit/test_kit.py para conferir o kit.

Passo 2: Organização e o Pull Request (PR) de Entrega

1. Para evitar conflitos de código entre os diferentes grupos da turma, toda a aplicação do seu grupo deve ficar dentro de uma subpasta exclusiva com o número do seu grupo. Alterações em /kit, /docs, no README da raiz ou em pastas de outros grupos não serão aceitas no PR.
    - O caminho do código no seu repositório deve ficar exatamente assim: /groups/group_XX/[your_code_here] (substitua o XX pelo número do seu grupo).
2. Após comitarem a versão final de **cada módulo** (até as 18h do prazo), abram um Pull Request direcionado à branch main do meu repositório central. Serão dois PRs: um para o Módulo 1 e outro para o Módulo 2.
3. Título do PR: Deve seguir o padrão: [Module X] - Group XX - Name1, Name2 and Name3 (ex.: [Module 1] - Group 03 - Ana, Bruno and Carla).

4. Como cada grupo estará em sua própria pasta, os Pull Requests não gerarão conflitos e servirão como o carimbo oficial de horário da entrega de vocês.

Passo 3: Deixem o "Star" (⭐) no Projeto

Para ajudar a impulsionar o repositório institucional da nossa disciplina na comunidade do GitHub, todos os alunos devem dar um **Star (⭐)** no repositório pai. Isso ajuda a ranquear o projeto e atrai relevância para o trabalho desenvolvido por vocês.

Boa sorte a todos no desenvolvimento do código!